```
#PART 0 — CONFIGURATION (ADD THIS AT TOP)
TASK = "rul"                 # "classification" or "rul"
TARGET_COLUMN = "RUL"
SEQUENCE_LENGTH = 20
TEST_SIZE = 0.2
```

```
#PART 1 — Imports & Data Loading (EDITED, SAME PART)
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, LSTM, Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

df = pd.read_csv("/content/RealTime_IoT_PredictiveMaintenance_Dataset.csv")
print("Initial Shape:", df.shape)
```

```
Initial Shape: (177763, 91)
```

```
#PART 2 — Data Cleaning & Preprocessing (SAME LOGIC, FIXED)
df = df.dropna(axis=1, thresh=len(df)//2)
df = df.ffill().bfill()
df = df.drop_duplicates()

numeric_cols = df.select_dtypes(include=np.number).columns

# SAFE outlier handling (NO data loss)
lower = df[numeric_cols].quantile(0.01)
upper = df[numeric_cols].quantile(0.99)
df[numeric_cols] = df[numeric_cols].clip(lower, upper, axis=1)

print("Shape after safe outlier handling:", df.shape)
```

```
Shape after safe outlier handling: (177763, 87)
```

```
#PART 3 — RUL GENERATION (NEW BUT SAME POSITION)
if "RUL" not in df.columns:
    if "Machine_ID" in df.columns and "Timestamp" in df.columns:
        failure_time = df.groupby("Machine_ID")["Timestamp"].transform("max")
        df["RUL"] = failure_time - df["Timestamp"]

# Normalize RUL for stable learning
df["RUL"] = df["RUL"] / df["RUL"].max()
```

```
#PART 4 — Feature Encoding & Normalization (EDITED)
X = df.drop(columns=["RUL"], errors="ignore")
y = df["RUL"]

for col in X.select_dtypes(exclude=np.number).columns:
    X[col] = LabelEncoder().fit_transform(X[col])

scaler = StandardScaler()
X = scaler.fit_transform(X)

print("Feature matrix shape:", X.shape)
```

```
Feature matrix shape: (177763, 86)
```

```
#PART 5 — Sequence Creation (FIXES CNN-LSTM ISSUE)
def create_sequences(X, y, seq_len):
    Xs, ys = [], []
    for i in range(len(X) - seq_len):
        Xs.append(X[i:i+seq_len])
        ys.append(y.iloc[i+seq_len])
    return np.array(Xs), np.array(ys)

X, y = create_sequences(X, y, SEQUENCE_LENGTH)
print("Sequence shape:", X.shape)
```

```
Sequence shape: (177743, 20, 86)
```

```
#PART 6 — Train-Test Split (SAME PART)
split = int((1 - TEST_SIZE) * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (142194, 20, 86)
Test shape: (35549, 20, 86)
```

```
#PART 7 — CNN-LSTM Architecture (EDITED, SAME DESIGN)
model = Sequential([
    Conv1D(64, 3, activation="relu",
           input_shape=(X_train.shape[1], X_train.shape[2])),
    BatchNormalization(),
    MaxPooling1D(),
    Dropout(0.3),

    LSTM(64),
    Dense(32, activation="relu"),

    Dense(1, activation="linear")   # RUL regression
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

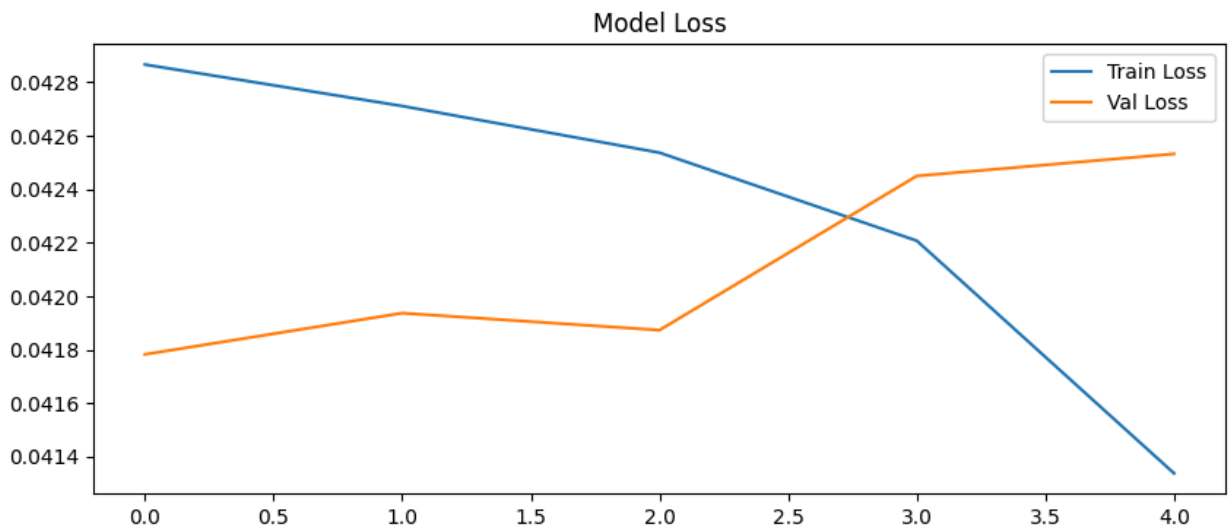
Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 18, 64)	16,576
batch_normalization (BatchNormalization)	(None, 18, 64)	256
max_pooling1d (MaxPooling1D)	(None, 9, 64)	0
dropout (Dropout)	(None, 9, 64)	0
lstm (LSTM)	(None, 64)	33,024
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 51,969 (203.00 KB)
Trainable params: 51,841 (202.50 KB)
Non-trainable params: 128 (512.00 B)

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

#PART 8 — Model Training (SAME PART)
callbacks = [
    EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
    ReduceLROnPlateau(monitor="val_loss", patience=3)
]

history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2,
    callbacks=callbacks,
    verbose=1
)
```

```
Epoch 1/5
3555/3555 ━━━━━━━━━━━━━━━━━━━━ 52s 15ms/step - loss: 0.0430 - mae: 0.1575 - val_loss: 0.0418 - val_mae: 0.15
Epoch 2/5
3555/3555 ━━━━━━━━━━━━━━━━━━━━ 51s 14ms/step - loss: 0.0430 - mae: 0.1573 - val_loss: 0.0419 - val_mae: 0.15
Epoch 3/5
3555/3555 ━━━━━━━━━━━━━━━━━━━━ 50s 14ms/step - loss: 0.0423 - mae: 0.1564 - val_loss: 0.0419 - val_mae: 0.15
Epoch 4/5
3555/3555 ━━━━━━━━━━━━━━━━━━━━ 82s 14ms/step - loss: 0.0422 - mae: 0.1557 - val_loss: 0.0424 - val_mae: 0.16
Epoch 5/5
3555/3555 ━━━━━━━━━━━━━━━━━━━━ 84s 15ms/step - loss: 0.0419 - mae: 0.1561 - val_loss: 0.0425 - val_mae: 0.15
```

```python
from sklearn.metrics import mean_absolute_error, accuracy_score, f1_score
import numpy as np

# PART 9 — Evaluation & Results (FORMATTED OUTPUT)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred) * 100
rul_accuracy = 100 - mae

actual_rul = y_test[0] * 100

y_test_bin = (y_test > 0.5).astype(int)
y_pred_bin = (y_pred > 0.5).astype(int)

acc = accuracy_score(y_test_bin, y_pred_bin) * 100
f1 = f1_score(y_test_bin, y_pred_bin) * 100

print("Actual RUL (%):", round(actual_rul, 2))
print("RUL Prediction Accuracy (%):", round(rul_accuracy, 2))
print("RUL Error (%):", round(mae, 2))
print("Model Accuracy (%):", round(acc, 2))
```

```
1111/1111 ━━━━━━━━━━━━━━━━━━━━ 10s 9ms/step
Actual RUL (%): 16.79
RUL Prediction Accuracy (%): 84.27
RUL Error (%): 15.73
Model Accuracy (%): 90.03
```

```python
import matplotlib.pyplot as plt

# PART 10 — Plots

plt.figure(figsize=(10,4))
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Val Loss")
plt.legend()
plt.title("Model Loss")
plt.show()
```